Python important Q&A

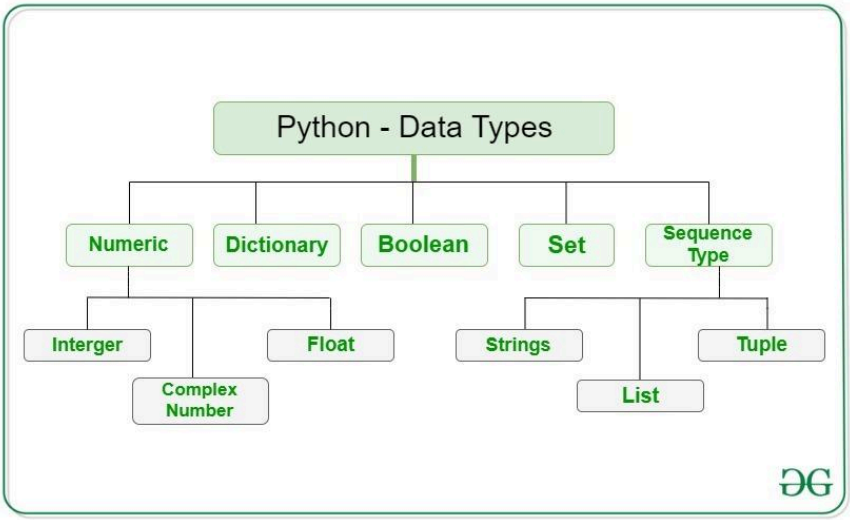
1.What is Python?

Python is a high-level, interpreted, and general-purpose programming language. It is known for its easy-to-read syntax and versatility, making it suitable for various applications, including web development, data analysis, and artificial intelligence.

2.What are the key features of Python?

- Easy syntax and readability
- Interpreted and dynamically typed
- Supports object-oriented, functional, and procedural programming
- Large standard library
- Cross-platform compatibility

3.What are Python's data types?



There are several built-in data types in Python, including:

- Numeric types: int, float, complex
- Sequence types: list, tuple, range

- Mapping type: dict
- Set types: set, frozenset
- Text type: str
- Boolean type: bool
- Binary types: bytes, bytearray, memoryview

4.What are lists and tuples in Python?

List in Python: Lists are mutable (modifiable) ordered collections in Python that can hold a variety of object types, including integers, strings, and other lists. They allow for dynamic resizing.

Tuples in Python: Tuples are immutable (non-modifiable) ordered collections in Python. Once created, the elements in a tuple cannot be changed, making them suitable for storing fixed data.

5.What is a dictionary in Python?

A dictionary in Python is a collection of key-value pairs where each key is unique. It is unordered, mutable, and indexed. It allows fast lookups and is defined using curly braces { }.

6.What are Lambda functions?

A lambda function in Python is a small anonymous function defined using the lambda keyword. It can take any number of arguments but only has one expression.

```
# A lambda function to find the square of a number

square = lambda x: x ** 2

print("Square of 5:", square(5))
```

Output Square of 5: 25

7.What is the difference between is and == in Python?

The difference between is and == in Python

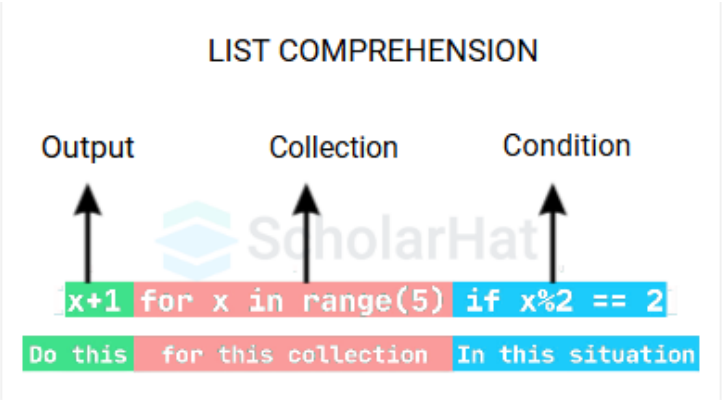
- == checks if the values of two objects are equal.
- is checks if two objects refer to the same memory location (i.e., they are the same object).

8.What is the difference between append() and extend() in Python?

The difference between append() and extend() in Python:

- append() adds a single element to the end of a list.
- extend() adds all elements from an iterable to the end of the list.

9.What are list comprehensions in Python?



List comprehensions in Python provide a concise way to create lists. They consist of an expression followed by a for clause, optionally including **if** conditions.

```
# List of numbers

numbers = [1, 2, 3, 4, 5]

# List comprehension to square each number

squared_numbers = [x ** 2 for x in numbers]

print("Squared Numbers:", squared_numbers)
```

Output

Squared Numbers: [1, 4, 9, 16, 25]

10.Is Python a compiled language or an interpreted language?

Please remember one thing, whether a language is compiled or interpreted or both is not defined in the language standard. In other words, it is not a properly of a programming language. Different Python distributions (or implementations) choose to do different things (compile or interpret or both). However the most common implementations like CPython do both compile and interpret, but in different stages of its execution process.

- **Compilation:** When you write Python code and run it, the source code (.py files) is first compiled into an intermediate form called bytecode (.pyc files). This bytecode is a lower-level representation of your code, but it is still not directly machine code. It's something that the Python Virtual Machine (PVM) can understand and execute.
- **Interpretation:** After Python code is compiled into bytecode, it is executed by the Python Virtual Machine (PVM), which is an interpreter. The PVM reads the bytecode and executes it line-by-line at runtime, which is why Python is considered an interpreted language in practice.

Some implementations, like PyPy, use Just-In-Time (JIT) compilation, where Python code is compiled into machine code at runtime for faster execution, blurring the lines between interpretation and compilation.

11.How can you concatenate two lists in Python?

We can concatenate two lists in Python using the +operator or the extend() method.

1. Using the + operator:

```
a = [1, 2, 3]

b = [4, 5, 6]

res = a + b

print(res)
```

2. Using the extend() method:

```
a = [1, 2, 3]

b = [4, 5, 6]

a.extend(b)

print(a)
```

12.Difference between for loop and while loop in Python

- For loop: Used when we know how many times to repeat, often with lists, tuples, sets, or dictionaries.
- While loop: Used when we only have an end condition and don't know exactly how many times it will repeat.

13.What is the difference between / and // in Python?

/ represents precise division (result is a floating point number) whereas // represents floor division (result is an integer). For Example:

```
print(5//2)

print(5/2)
```

Output

```
2
2.5
```

14.What is a dynamically typed language?

- In a dynamically typed language, the data type of a variable is determined at runtime, not at compile time.
- No need to declare data types manually; Python automatically detects it based on the assigned value.
- Examples of dynamically typed languages: Python, JavaScript.
- Examples of statically typed languages: C, C++, Java.
- Dynamically typed languages are easier and faster to code.
- Statically typed languages are usually faster to execute due to type checking at compile time.

15.What is the difference between a Set and Dictionary?

- A Python Set is an unordered collection data type that is iterable, mutable and has no duplicate elements. Python's set class represents the mathematical notion of a set.
- Syntax: Defined using curly braces {} or the set() function.

```
my_set = {1, 2, 3}
```

- Dictionary in Python is an ordered (since Py 3.7) [unordered (Py 3.6 & prior)] collection of data values, used to store data values like a map, which, unlike other Data Types that hold only a single value as an element, Dictionary holds key:value pair. Key-value is provided in the dictionary to make it more optimized.
- Syntax: Defined using curly braces {} with key-value pairs.

```
my_dict = {"a": 1, "b": 2, "c": 3}
```

16.What are Built-in data types in Python?

The following are the standard or built-in data types in Python:

- Numeric: The numeric data type in Python represents the data that has a numeric value. A numeric value can be an integer, a floating number, a Boolean, or even a complex number.
- Sequence Type: The sequence Data Type in Python is the ordered collection of similar or different data types. There are several sequence types in Python:
    - Python String
    - Python List
    - Python Tuple
    - Python range
- Mapping Types: In Python, hashable data can be mapped to random objects using a mapping object. There is currently only one common mapping type, the dictionary and mapping objects are mutable.
    - Python Dictionary
- Set Types: In Python, a Set is an unordered collection of data types that is iterable, mutable and has no duplicate elements. The order of elements in a set is undefined though it may consist of various elements.

17. What is the difference between a Mutable datatype and an Immutable data type?

- Mutable data types can be edited i.e., they can change at runtime. Eg – List, Dictionary, etc.
- Immutable data types can not be edited i.e., they can not change at runtime. Eg – String, Tuple, etc.

18.What is a Variable Scope in Python?

The location where we can find a variable and also access it if required is called the scope of a variable.

- Python Local variable: Local variables are those that are initialized within a function and are unique to that function. A local variable cannot be accessed outside of the function.
- Python Global variables: Global variables are the ones that are defined and declared outside any function and are not specified to any function.
- Module-level scope: It refers to the global objects of the current module accessible in the program.
- Outermost scope: It refers to any built-in names that the program can call. The name referenced is located last among the objects in this scope.

19. How is a dictionary different from a list?

A list is an ordered collection of items accessed by their index, while a dictionary is an unordered collection of key-value pairs accessed using unique keys. Lists are ideal for sequential data, whereas dictionaries are better for associative data. For example, a list can store [10, 20, 30], whereas a dictionary can store {"a": 10, "b": 20, "c": 30}.

20. What is docstring in Python?

Python documentation strings (or docstrings) provide a convenient way of associating documentation with Python modules, functions, classes and methods.

- Declaring Docstrings: The docstrings are declared using ”’triple single quotes”’ or “”””triple double quotes””” just below the class, method, or function declaration. All functions should have a docstring.
- Accessing Docstrings: The docstrings can be accessed using the __doc__ method of the object or using the help function.

21.What is the difference between Python Arrays and Lists?

- Arrays (when talking about the array module in Python) are specifically used to store a collection of numeric elements that are all of the same type. This makes them more efficient for storing large amounts of data and performing numerical computations where the type consistency is maintained.
- Syntax: Need to import the array module to use arrays.

Example:

```
from array import array
arr = array('i', [1, 2, 3, 4])  # Array of integers
print (arr)
```

```
array('i', [1, 2, 3, 4])
```

Output

- Lists are more flexible than arrays in that they can hold elements of different types (integers, strings, objects, etc.). They come built-in with Python and do not require importing any additional modules.
- Lists support a variety of operations that can modify the list.

Example:

```
a = [1, 'hello', 3.14, [1, 2, 3]]
print (a)
```

```
[1, 'hello', 3.14, [1, 2, 3]]
```

22.Differentiate between List and Tuple?

Let's analyze the differences between List and Tuple:

List

- Lists are Mutable datatype.

- Lists consume more memory
- The list is better for performing operations, such as insertion and deletion.
- The implication of iterations is Time-consuming

Tuple

- Tuples are Immutable datatype.
- Tuple consumes less memory as compared to the list
- A Tuple data type is appropriate for accessing the elements
- The implication of iterations is comparatively Faster

23.What is slicing in Python?

Python Slicing is a string operation for extracting a part of the string, or some part of a list. With this operator, one can specify where to start the slicing, where to end and specify the step. List slicing returns a new list from the existing list.

Syntax: substring = s[start : end : step]

Example:

```
b = "Hello, World!"
print(b[2:5])
o.p: llo


b = "Hello, World!"
print(b[:5])
O.p: Hello


b = "Hello, World!"
print(b[2:])
O.p:llo, World!
```

24.What is break, continue and pass in Python?

The break statement terminates the loop immediately and the control flows to the statement after the body of the loop.

The continue statement terminates the current iteration of the statement, skips the rest of the code in the current iteration and the control flows to the next iteration of the loop.
The pass keyword in Python is generally used to fill up empty blocks and is similar to an empty statement represented by a semi-colon in languages such as Java, C++, Javascript, etc.

```
pat = [1, 3, 2, 1, 2, 3, 1, 0, 1, 3]
for p in pat:
    pass
    if (p == 0):
        current = p
        break
    elif (p % 2 == 0):
        continue
    print(p)    # output => 1 3 1 3 1
print(current)    # output => 0
```

25. What are negative indexes and why are they used?

- Negative indexes are the indexes from the end of the list or tuple or string.
- Arr[-1] means the last element of array Arr[]

```
arr = [1, 2, 3, 4, 5, 6]
#get the last element
print(arr[-1]) #output 6
#get the second last element
print(arr[-2]) #output 5
```

26.Explain split() and join() functions in Python?

- You can use split() function to split a string based on a delimiter to a list of strings.
- You can use join() function to join a list of strings based on a delimiter to give a single string.

```
string = "This is a string."
string_list = string.split(' ') #delimiter is ‘space’ character or ‘ ‘
print(string_list) #output: ['This', 'is', 'a', 'string.']
print(' '.join(string_list)) #output: This is a string.
```